

Toward Efficient Deployment of Mixture of Experts Models: Quantization and Compression Analysis

Dongjun Lee, Jin Shin, and Hyun Kim

Department of Electrical and Information Engineering and Research Center for Electrical and Information Technology

Seoul National University of Science and Technology

Seoul 01811, Korea

dj.lee@seoultech.ac.kr, shinjin0103@seoultech.ac.kr, hyunkim@seoultech.ac.kr

Abstract—Recent mixture of experts (MoE)-based transformer models have achieved remarkable performance across various applications. However, as model size increases, the associated memory demand escalates dramatically, limiting their deployment in resource-constrained edge environments. To address this challenge, this study investigates the trade-off between performance and computational overhead under extreme compression conditions by applying ternary quantization and additional compression schemes to MoE models. Experimental results demonstrate that Huffman coding achieves a higher compression ratio but suffers from substantial decoding latency, while dictionary coding provides a lower compression ratio with significantly faster decoding. These findings highlight that efficient deployment of MoE models requires an integrated compression framework that jointly optimizes compression ratio and overhead. Furthermore, based on these observations, potential research directions are suggested, including hybrid coding formats and look-up table-based dedicated hardware accelerators for efficient MoE model compression.

Index Terms—Mixture of Experts, Ternary Quantization, Faster Decoding, Dedicated Hardware Accelerators, Edge Deploy

I. INTRODUCTION

Transformer-based architectures [1], [2] have demonstrated outstanding performance across a wide range of applications [3], [4]. However, as model size increases, both computational complexity and memory consumption grow exponentially, leading to substantial computational costs during training and inference [5]–[7]. To mitigate these challenges, the mixture of experts (MoE) architecture has been proposed [8]–[10], which enhances computational efficiency by selectively activating a subset of experts for each input token. Although this selective activation prevents a significant increase in computational load, the total number of model parameters still grows considerably, resulting in a much larger memory footprint compared to dense Transformer models. Consequently, deploying MoE models in memory-constrained environments, such as single GPUs or edge devices, remains a substantial challenge [11]–[13]. To address this issue, various model compression techniques—particularly quantization [14]–[16]

and pruning [17], [18]—have been actively explored. However, these methods still face limitations in reducing memory usage to a level suitable for deployment without causing significant accuracy degradation. Consequently, several studies have proposed applying additional compression schemes to quantized models [19]. Nevertheless, such approaches often introduce decoding overhead, which can degrade real inference efficiency. Therefore, it is essential to analyze the trade-off between compression efficiency and decoding overhead and to develop optimized compression strategies for MoE-based models that balance these competing factors.

Accordingly, this study applies dictionary-based coding and Huffman coding to a model quantized using ternary quantization, and compares the compression ratio, memory reduction, and decoding time of each method. Based on the analysis results, we further suggest future research directions for efficient MoE compression techniques that take into account both computational characteristics and memory access patterns.

II. BACKGROUNDS

A. Ternary Quantization

Quantization is a widely adopted model compression technique that reduces memory consumption by representing data with lower bit precision. In particular, ternary quantization constrains data values to three discrete levels, thereby achieving an extreme reduction in bit width. However, this approach can introduce considerable quantization error; thus, methods such as clipping and scaling factor adjustment are employed to mitigate this error.

In this study, we compare round-to-nearest (RTN) and GPTQ [20]-based ternary quantization to evaluate their relative performance. RTN quantizes real-valued parameters by rounding them to the nearest discrete level, whereas GPTQ minimizes quantization loss by leveraging second-order Hessian information to compensate for quantization errors more effectively.

B. Compression Background

In this study, we compare two additional compression methods applied to quantized weights: dictionary encoding and Huffman encoding. The dictionary-based approach compresses repeated weight patterns by storing them in a fixed-length dictionary structure, thereby eliminating redundancy. In

This research was partly supported by Basic Science Research Program through the National Research Foundation of Korea(NRF) funded by the Ministry of Education(RS-2025-25429747) and K-CHIPS (Korea Collaborative & High-tech Initiative for Prospective Semiconductor Research) (RS-2025-02305531) funded by the Ministry of Trade, Industry & Energy (MOTIE, Korea).

contrast, Huffman encoding is a variable-length entropy coding technique that assigns shorter codes to frequently occurring symbols and longer codes to less frequent ones, optimizing overall bit efficiency.

III. EXPERIMENTS

A. Experimental Settings

All experiments in this study were conducted using the Switch-Transformer Base-128 model as a representative MoE architecture. The experiments were performed on a single NVIDIA A100 GPU (80 GB) using the PyTorch framework. The ternary quantization scheme was implemented following the methodology proposed in [19], and evaluations were carried out on the C4 [21] and WikiText2 [22] datasets. To comprehensively analyze the effects of additional compression techniques, we measured GPU memory consumption, module-wise memory distribution, and decoding latency during inference.

B. Memory Distribution of Each Module

We evaluated the module-wise memory allocation of the Switch-Transformer Base-128 model to identify the primary contributors to overall memory consumption. Table I summarizes the memory usage of the Embedding, Attention, and Expert modules, while the remaining auxiliary components are aggregated under the *Others* column. As shown in Table I, the expert modules occupy 13.92 GB out of the total 14.28 GB model size, accounting for approximately 97.4% of the overall memory footprint. This is because, in the MoE architecture, all expert parameters must be stored simultaneously, even though only a subset of experts is activated during inference. Consequently, achieving memory efficiency in MoE models requires the application of aggressive lightweighting techniques specifically targeting the expert modules. To address this issue, we experimentally analyzed the effectiveness of ternary quantization and additional compression methods in mitigating the expert memory bottleneck.

C. Quantization Result

Table II presents the results of various quantization methods applied to the Switch-Transformer Base-128 model. The baseline represents the original model computed with half-precision floating-point (FP16) precision, while W-bit and A-bit denote the bit precision of the weights and activations, respectively. To achieve an extreme reduction in memory usage, 2-bit and ternary quantization schemes were applied, resulting in more than an $8\times$ reduction in memory consumption compared to the FP16 baseline. For the RTN method, a slight increase in loss was observed with 2-bit quantization, whereas ternary quantization led to a more significant degradation. In contrast, GPTQ maintained stable performance, exhibiting only a minor increase in loss even under ternary quantization.

TABLE I
MEMORY DISTRIBUTION OF MAJOR MODULES IN THE FP16 BASELINE MODEL.

Module	Embedding	Attention	Expert	Others	Total
Size (GB)	0.09	0.16	13.92	0.11	14.28

TABLE II
PERFORMANCE COMPARISON OF TERNARY QUANTIZED MODELS IN TERMS OF LOSS ON C4 AND WIKITEXT2 DATASETS.

Method	W-bit	A-bit	C4 ($\downarrow$)	WikiText2 ($\downarrow$)
Baseline	16	16	1.73	1.97
RTN	2	16	2.27	2.43
GPTQ	2	16	1.82	1.99
RTN	1.58	16	4.54	4.27
GPTQ	1.58	16	2.39	2.27

D. Compression on Quantized Weights

Table III summarizes the results of applying additional compression techniques to the ternary-quantized weights. The baseline corresponds to the original FP16-precision model. A dictionary denotes a fixed-length compression scheme based on dictionary coding, whereas Huffman represents a variable-length entropy-based approach. In this context, Size indicates the total model size, and Compression ratio is defined as (FP16 model size)/(Compressed model size), reflecting the compression ratio relative to the baseline. Bit/Weight denotes the average number of bits required to represent a single weight, and Time reports the decoding latency of the compressed model. As shown in Table III, applying dictionary coding after ternary quantization achieved an $11.96\times$ compression ratio compared to the FP16 model, whereas Huffman coding achieved a higher $13.49\times$ ratio due to its variable-length encoding property. However, the two methods exhibited a substantial difference in decoding latency: dictionary coding required only 0.72 s, while Huffman decoding incurred a significantly larger latency of 2479.43 s.

IV. OVERALL ANALYSIS AND DISCUSSION

A summary of the experimental results presented in Section III is as follows. As shown in Table I, the majority of the memory usage in the MoE model is concentrated in the Expert modules, indicating that applying aggressive compression or quantization to the Expert component is essential for improving model efficiency and enabling practical deployment.

In Section III-D, the results of applying additional compression methods after ternary quantization demonstrate that while Huffman coding achieved a higher compression ratio, it incurred substantially greater decoding latency compared to dictionary coding. This is because dictionary coding employs a fixed-length format that allows for efficient parallel decoding, whereas Huffman coding relies on variable-length encoding, which requires the completion of each preceding code before the next can be decoded—a fundamental structural limitation.

TABLE III
COMPARISON OF COMPRESSION METHODS APPLIED TO
TERNARY-QUANTIZED WEIGHTS.

Method	Size (GB)	Comp. Ratio	Bit / Weight	Time (s)
Baseline	14.28	1.00×	16.00	—
Dictionary	1.19	11.96×	1.34	0.72
Huffman	1.06	13.49×	1.19	2479.43

Moreover, the decoding latency of Huffman coding is expected to increase further as the model size scales.

Therefore, achieving efficient compression for MoE-based models requires considering not only the compression ratio but also the associated decoding latency. Motivated by this observation, a hybrid coding format that performs frequency-based encoding within a fixed-length representation could potentially enhance compression efficiency while reducing decoding delay. Furthermore, by leveraging the properties of ternary quantization, designing a look-up table (LUT)-based dedicated hardware operator for the proposed compression format could reduce hardware resource utilization and computational complexity, while simultaneously minimizing the overhead introduced during the decompression process.

V. CONCLUSION

In this study, we applied ternary quantization and compression techniques to MoE-based models to analyze the trade-off between compression efficiency and computational overhead. The experimental results demonstrate that efficient deployment of MoE models requires an integrated design that accounts not only for compression ratio but also for the associated decoding overhead. In conclusion, this work quantitatively analyzed the trade-off between model performance and latency induced by compression, and further suggests future research directions such as developing hybrid coding formats and LUT-based dedicated hardware accelerators for more efficient MoE model compression.

REFERENCES

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.
- [2] S. I. Lee, K. Koo, J. H. Lee, G. Lee, S. Jeong, S. O, and H. Kim, "Vision transformer models for mobile/edge devices: a survey," *Multimedia Systems*, vol. 30, no. 2, p. 109, 2024.
- [3] N. J. Kim, J. Lee, and H. Kim, "Hyq: Hardware-friendly post-training quantization for cnn-transformer hybrid networks," in *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, IJCAI-24. International Joint Conferences on Artificial Intelligence Organization*, vol. 8, 2024, pp. 4291–4299.
- [4] B. J. Kang, N. Kim, and H. Kim, "Lra-qvit: Integrating low-rank approximation and quantization for robust and efficient vision transformers," in *Forty-second International Conference on Machine Learning*.
- [5] S. Jeong, S. Lee, and H. Kim, "Lowgradq: Adaptive gradient quantization for low-bit cnn training via kernel density estimation-guided thresholding and hardware-efficient stochastic rounding unit," in *2025 Design, Automation & Test in Europe Conference (DATE)*. IEEE, 2025, pp. 1–2.
- [6] D. Choi and H. Kim, "Gradq-vit: Robust and efficient gradient quantization for vision transformers," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 15, 2025, pp. 16019–16027.

- [7] B. J. Kang, H. I. Lee, S. K. Yoon, Y. C. Kim, S. B. Jeong, and H. Kim, "A survey of fpga and asic designs for transformer inference acceleration and optimization," *Journal of Systems Architecture*, vol. 155, p. 103247, 2024.
- [8] A. Q. Jiang, A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chaplot, D. d. l. Casas, E. B. Hanna, F. Bressand *et al.*, "Mixtral of experts," *arXiv preprint arXiv:2401.04088*, 2024.
- [9] D. Dai, C. Deng, C. Zhao, R. Xu, H. Gao, D. Chen, J. Li, W. Zeng, X. Yu, Y. Wu *et al.*, "Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models," *arXiv preprint arXiv:2401.06066*, 2024.
- [10] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv *et al.*, "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025.
- [11] M. Ai, T. Wei, Y. Chen, Z. Zeng, R. Zhao, G. Varatkar, B. D. Rouhani, X. Tang, H. Tong, and J. He, "Resmoe: Space-efficient compression of mixture of experts llms via residual restoration," in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1*, 2025, pp. 1–12.
- [12] R. Sarkar, H. Liang, Z. Fan, Z. Wang, and C. Hao, "Edge-moe: Memory-efficient multi-task vision transformer architecture with task-level sparsity via mixture-of-experts," in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 2023, pp. 01–09.
- [13] P. Jin, B. Zhu, L. Yuan, and S. YAN, "Moe++: Accelerating mixture-of-experts methods with zero-computation experts," in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: <https://openreview.net/forum?id=7P5BUKcYv>
- [14] Z. Chen, X. Hu, D. Yang, Z. Xu, XUCHEN, Z. Yuan, S. Zhou, and JiangyongYu, "MoEQuant: Enhancing quantization for mixture-of-experts large language models via expert-balanced sampling and affinity guidance," in *Forty-second International Conference on Machine Learning*, 2025. [Online]. Available: <https://openreview.net/forum?id=0epuNvt5Dj>
- [15] H. Duanmu, X. Li, Z. Yuan, S. Zheng, J. Duan, X. Zhang, and D. Lin, "Mxmoe: Mixed-precision quantization for moe with accuracy and performance co-design," in *Forty-second International Conference on Machine Learning*, 2025. [Online]. Available: <https://openreview.net/forum?id=pXoZLGMNDm>
- [16] B. Huang, Y. Yuan, Z. SHAO, and M. Zhang, "Milo: Efficient quantized moe inference with mixture of low-rank compensators," in *Eighth Conference on Machine Learning and Systems*, 2025. [Online]. Available: <https://openreview.net/forum?id=NXVXiJmhe1>
- [17] E. Liu, J. Zhu, Z. Lin, X. Ning, M. B. Blaschko, S. Yan, G. Dai, H. Yang, and Y. Wang, "Efficient expert pruning for sparse mixture-of-experts language models: Enhancing performance and reducing inference costs," *arXiv preprint arXiv:2407.00945*, 2024.
- [18] Y. Xie, Z. Zhang, D. Zhou, C. Xie, Z. Song, X. Liu, Y. Wang, X. Lin, and A. Xu, "Moe-pruner: Pruning mixture-of-experts large language model using the hints from its router," *arXiv preprint arXiv:2410.12013*, 2024.
- [19] E. Frantar and D. Alistarh, "Qmoe: Sub-1-bit compression of trillion parameter models," *Proceedings of Machine Learning and Systems*, vol. 6, pp. 439–451, 2024.
- [20] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "OPTQ: Accurate quantization for generative pre-trained transformers," in *The Eleventh International Conference on Learning Representations*, 2023. [Online]. Available: <https://openreview.net/forum?id=tcbBPnfwxS>
- [21] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of machine learning research*, vol. 21, no. 140, pp. 1–67, 2020.
- [22] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer sentinel mixture models," in *International Conference on Learning Representations*, 2017. [Online]. Available: <https://openreview.net/forum?id=Byj72udxe>